

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO

Bases de Datos I · SD1006 · Grupo 811

SOLARDB PASCUAL: GOBERNANZA DE DATOS, AUTOMATIZACIÓN ETL E IoT

Trabajo de consulta en parejas y práctica guiada

Integrante 1: Sebastián Ceballos Garzón

Integrante 2: Juan Pablo Barrientos Lopez

Docente: Ramiro Grisales Montoya

Semestre: 2026-II

Fecha: 4 de octubre de 2026 .

1. Parte A · Consulta

Las respuestas se aplican al caso SolarDB Pascual y se redactan en palabras propias. Las fuentes se identifican mediante referencias numeradas al final. El alcance se limita a cómo se gobierna, transporta y carga el dato en PostgreSQL, sin incluir tableros.

G1. Gobernanza de datos

Respuesta. La gobernanza de datos es el conjunto de responsabilidades, reglas, decisiones y mecanismos de supervisión que determinan quién puede decidir sobre los datos, bajo qué políticas y con qué criterios de control. DAMA-DMBOK la ubica dentro de la disciplina de gestión de datos y la relaciona con responsabilidad, políticas y derechos de decisión. Data management es más amplio:

comprende el desarrollo, ejecución y supervisión de planes y prácticas que entregan, controlan, protegen y mejoran el valor de los datos durante su ciclo de vida. Data quality se enfoca en que los datos sean confiables para su uso. En SolarDB, gobernanza define responsables y reglas; management implementa los procesos y tecnología; quality verifica que las lecturas cumplan las reglas.

Fuente: [1].

G2. Roles data owner, data steward y data custodian

Respuesta. El data owner tiene la responsabilidad de negocio sobre el dato y decide sus reglas de uso y acceso. El data steward administra el significado, calidad, metadatos y seguimiento cotidiano de esas reglas. El data custodian se ocupa de la implementación técnica, almacenamiento, seguridad y operación. Para SolarDB proponemos como owner al responsable académico o funcional del proyecto SolarDB Pascual; como steward, al integrante encargado de documentar el diccionario, calidad y trazabilidad; y como custodian, al integrante responsable de PostgreSQL, ETL, copias y permisos. Esta división evita que una sola persona tome todas las decisiones y permite relacionar la responsabilidad funcional con la técnica.

Fuente: [1].

G3. Metadatos y catálogo de datos

Respuesta. Los metadatos son datos que describen otros datos: nombre de una columna, tipo, significado, origen, unidad, frecuencia o regla de validación. Un catálogo de datos reúne y organiza esos metadatos para facilitar descubrimiento, comprensión, gobierno y trazabilidad. En el diccionario de SolarDB, para convertirlo en un catálogo gobernado convendría agregar, por cada atributo, al menos: responsable, origen, regla de calidad y fecha de actualización. También es útil conservar unidad de medida, sensibilidad, periodicidad y sistema fuente. Por ejemplo, p_ac podría tener origen “inversor”, unidad kW, regla “>= 0”, responsable “steward ETL” y fecha de actualización. Así el diccionario deja de ser solo una descripción técnica y pasa a servir para control y responsabilidad.

Fuente: [1].

G4. Dimensiones de calidad y restricciones PostgreSQL

Respuesta. Completitud significa que no falten valores necesarios; validez, que respeten formato o rango; unicidad, que no existan duplicados donde el negocio exige uno; consistencia, que los datos mantengan relaciones compatibles; oportunidad, que lleguen en el tiempo esperado; y exactitud, que representen correctamente la realidad. En SolarDB se pueden traducir así: **NOT NULL** para timestamp y dispositivo; **CHECK** para impedir p_ac negativo o irradiancia fuera del rango; **UNIQUE/PRIMARY KEY** para impedir dos lecturas del mismo dispositivo en el mismo instante; y **FOREIGN KEY** para relacionar una lectura con un dispositivo registrado. Estas restricciones convierten reglas de calidad en controles automáticos dentro de la base.

Fuente: [1] y diseño de la práctica del documento.

G5. Mínimo privilegio y roles

Respuesta. El principio de mínimo privilegio indica que cada cuenta debe recibir solamente los permisos indispensables para cumplir su función. En SolarDB se proponen tres roles: **solar_ingesta**, para insertar datos de staging y consultar lo necesario para el proceso; **solar_lector**, exclusivamente para consultar lectura_demo; y **solar_admin**, para administrar tablas y permisos. El rol de lectura no debe tener INSERT, UPDATE ni DELETE. Los permisos se asignan explícitamente con GRANT y se eliminan privilegios innecesarios con REVOKE. Esto reduce el impacto de errores o credenciales comprometidas y hace más clara la responsabilidad sobre cada operación.

Rol	stg_lectura_raw	lectura_demo	etl_log
solar_ingesta	SELECT, INSERT	—	—
solar_lector	—	SELECT	—
solar_admin	SELECT, INSERT, UPDATE, DELETE	SELECT, INSERT, UPDATE, DELETE	SELECT, INSERT, UPDATE, DELETE

Fuente: [2], [3], [4].

G6. Ley 1581 de 2012 y postura ética

Respuesta. La telemetría técnica de una planta —por ejemplo potencia, irradiancia o temperatura— no es, por sí misma, información vinculada a una persona natural determinada o determinable; por eso la consideramos inicialmente dato técnico/no personal. La situación cambia si SolarDB registra operadores asociados a alarmas, porque nombre, identificación, contacto, turnos o acciones pueden constituir datos personales y quedar sujetos a la Ley 1581. Nuestra postura ética es aplicar finalidad, minimización, acceso restringido, trazabilidad y protección de la información. Si se registra al operador, solo deben conservarse los atributos necesarios para atender la finalidad definida y los accesos deben limitarse según la función. La protección no debe depender únicamente de que el dato parezca operativo.

Fuente: [5].

E1. ETL frente a ELT

Respuesta. ETL significa extraer, transformar y cargar: la transformación ocurre antes de que el dato llegue al repositorio destino. ELT carga primero el dato y transforma después dentro del sistema de almacenamiento. El flujo planteado para SolarDB se parece más a **ELT**

porque el mensaje crudo del gateway se conserva primero en `stg_lectura_raw.payload` como JSONB y después SQL extrae y convierte sus atributos para insertarlos en `lectura_demo`. Esta decisión conserva evidencia del mensaje original y permite repetir la transformación sin depender de un formato intermedio externo. En una implementación real podría existir una etapa ETL anterior en el gateway, pero la práctica concreta descrita en el documento tiene una transformación posterior a la carga en PostgreSQL.

Fuente: diseño de la práctica del documento y [6].

E2. Full load, incremental e idempotencia

Respuesta. Una carga completa vuelve a procesar todo el conjunto disponible; una incremental procesa únicamente datos nuevos o modificados desde la última ejecución. Una carga es idempotente cuando repetir la misma operación con los mismos datos deja el sistema en el mismo estado lógico, sin duplicaciones. En SolarDB, la clave primaria compuesta (`dispositivo_id`, `ts`) identifica una lectura. Al ejecutar `INSERT ... ON CONFLICT (dispositivo_id, ts) DO NOTHING`, una fila ya existente no se vuelve a insertar. PostgreSQL documenta que `ON CONFLICT` permite elegir una acción alternativa ante una violación de una restricción única; `DO NOTHING` evita insertar la fila conflictiva. Por ello, el segundo procesamiento del mismo archivo mantiene el total de lecturas en 288.

Fuente: [7].

E3. Programación de cargas

Forma	Qué es	Cuándo conviene	Complejidad
		tareasPlanificador del sistema operativo que ejecuta definidos. Prácticas pequeñas, scripts independientes y ajaja	
pg_cron		QL para programar trabajos dentro del motor. Cuando ción están muy ligadas a PostgreSQL. Media	
Apache Airflow		y orquestar flujos mediante DAGs, tareas y s con varias etapas, dependencias, reintentos y	

Elección: para esta práctica usaríamos `cron` en Linux o el Programador de tareas de Windows porque el flujo es corto y tiene un único punto de entrada. Airflow sería más apropiado cuando SolarDB incorpore varios procesos dependientes y necesite orquestación más completa.

Fuente: [8].

E4. Bitácora y gobernanza

Respuesta. La bitácora debe registrar, como mínimo, inicio, fin, filas leídas, filas cargadas, filas rechazadas o conflictivas, estado y mensaje de error. En esta práctica la tabla `etl_log` guarda esos datos por ejecución. La bitácora también es una herramienta de gobernanza porque permite responder qué proceso corrió, cuándo lo hizo, cuánto procesó y si falló. Esa información aporta trazabilidad o data lineage: conecta el archivo de origen con el staging, la transformación y la tabla destino. También facilita auditoría, diagnóstico de incidentes y medición de calidad del proceso.

Fuente: aplicación de trazabilidad al flujo SolarDB.

I1. Arquitectura IoT por capas

Respuesta. En SolarDB se propone una arquitectura de cinco capas. La capa de dispositivo contiene inversor, medidor y piranómetro; allí se producen las mediciones. El gateway o edge recibe y normaliza mensajes cerca de la planta. La red/broker transporta mensajes y permite desacoplar productores y consumidores. La capa de almacenamiento persiste los datos, en este caso PostgreSQL/SolarDB. Finalmente, la aplicación consulta los datos para operación o análisis. El inversor genera potencia, temperatura y alarmas; el medidor aporta variables eléctricas; el piranómetro aporta irradiancia. El gateway reúne la información y la envía al broker o directamente al proceso de ingesta. SolarDB conserva el dato crudo y el dato relacional transformado.



Flujo: captura → procesamiento en borde → transporte → persistencia → consumo.

Fuente: arquitectura propuesta para la práctica; MQTT se sustenta en [9].

12. MQTT y jerarquía de topics

Respuesta. MQTT usa un modelo publish/subscribe: un cliente publica mensajes y otros clientes se suscriben mediante un broker, que distribuye los mensajes según el topic. Un topic identifica el canal lógico del mensaje. QoS 0 ofrece "como máximo una vez", QoS 1 "al menos una vez" y QoS 2 "exactamente una vez" en el protocolo MQTT. Para SolarDB proponemos: pascualbravo/solar/<sitio>/<dispositivo>/<variable>. Ejemplo: pascualbravo/solar/pascual/inversor01/p_ac. Esta jerarquía permite suscripciones amplias o específicas y mantiene separados sitio, dispositivo y variable.

Fuente: [9].

13. Modbus, MQTT y OPC UA

Protocolo	Modelo de comunicación	Uso típico	Formato de datos	Seguridad
Modbus RTU/TCP				e/servidor según implementación; RTU P. Automatización industrial y otros. Registros y códigos de función; dbus clásico no ofrece por sí solo las
MQTT	Publish/subscribe con broker.			s con dispositivos distribuidos. Payload ; comúnmente JSON. TLS puede tenticación/autorización d
OPC UA	Cliente/servidor y también PubSub.			al y acceso a información de máquinas. y encodings como UA Binary/JSON. enticación, cifrado, firma y auditoría.

Fuentes: [9], [10], [11].

14. Payload JSON del inversor

Ejemplo de mensaje cada cinco minutos:

```
{
  "device_id": 1,
  "ts": "2026-10-05T06:00:00-05:00",
  "p_ac": 3.214,
  "irradiancia": 742.5,
  "temp_modulo": 36.8,
  "frecuencia_hz": 60.01,
  "alarma": "GRID_FAULT"
}
```

Unidades: **p_ac** en kW; **irradiancia** en W/m²; **temp_modulo** en °C; **frecuencia_hz** en Hz; **ts** en fecha/hora con zona horaria; **device_id** es identificador del dispositivo; **alarma** es opcional.

Consulta 1 — extracción de atributos:

```
SELECT
  payload --> 'device_id' AS dispositivo,
  payload --> 'p_ac' AS potencia_kw,
  payload --> 'ts' AS instante
FROM stg_lectura_raw;
```

Consulta 2 — operadores JSONB:

```
SELECT payload
FROM stg_lectura_raw
WHERE payload ? 'alarma'
AND payload @> '{"alarma":"GRID_FAULT"}';
```

Fuente: [6].

I5. Volumen mensual y almacenamiento

Estimación. 10 dispositivos × 5 variables × 60 lecturas/hora × 24 horas/día × 30 días = **2.160.000 filas/mes**. El particionamiento por rango puede separar la tabla por periodos de tiempo, por ejemplo meses, haciendo más manejables grandes volúmenes y permitiendo trabajar con las particiones relevantes. Un índice BRIN es adecuado para columnas cuyos valores están correlacionados con el orden físico de almacenamiento, como un timestamp en una tabla de series temporales; su estructura es pequeña frente a índices tradicionales y puede acelerar búsquedas por rangos. TimescaleDB es una alternativa especializada para series temporales sobre PostgreSQL cuando se necesiten funciones adicionales de este tipo de carga.

V1. Git y GitHub

Respuesta. Git es el sistema de control de versiones que registra cambios en los archivos; GitHub es una plataforma que aloja repositorios Git y añade colaboración, permisos, revisión y otros servicios. Un repository es el proyecto versionado; un commit es una fotografía lógica de cambios; una branch es una línea de trabajo; un pull request propone integrar cambios; y .gitignore define archivos que Git no debe rastrear. Nunca se deben subir contraseñas ni cadenas de conexión porque el historial puede conservarlas incluso después de borrarlas. Se evita mediante variables de entorno, un archivo .env ignorado y un .env.example sin secretos. En este repositorio se aplica esa separación.

Fuente: práctica de control de versiones del documento y buenas prácticas de Git.

2. Parte B · Práctica guiada

La práctica está preparada para PostgreSQL 15 o superior. El código completo se encuentra en el repositorio/paquete entregable; el PDF muestra únicamente los fragmentos necesarios para explicar el flujo.

Paso 1 · Simulador IoT

El simulador fue modificado para generar **dos dispositivos** (1 y 2) y añadir el campo nuevo **frecuencia_hz**. Cada dispositivo genera 144 mensajes, uno cada cinco minutos durante 12 horas, para un total de **288 mensajes**. Se mantiene la zona horaria de Colombia y el campo de alarma opcional.

Captura que debe insertarse aquí: ejecución de python etl/simulador.py y evidencia de que se generó data/lecturas.jsonl.

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Extensions
- Foreign Data Wrappers
- Languages
- Publications
- Schemas (1)
 - public
- Subscriptions
- postgres
- Login/Group Roles
- Tablespaces
- SolarDB - Lector
 - Databases (2)
 - SolarDB
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - postgres
 - Login/Group Roles
 - Tablespaces

SQL Statistics Dependencies Dependents Processes cargar_staging.sql* SolarDB/solar_lect...

SolarDB/postgres@PostgreSQL 18

No limit

Query Query History Scratch Pad

```
1 SELECT COUNT(*) AS total_lecturas
2 FROM stg_lectura_raw;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

total_lecturas	bigint
1	288

Total rows: 1 Query complete 00:00:00.085 CRLF Ln 2, Col 22

21°

ESP LAA 7:07 p.m. 4/10/2026

Paso 2 · Staging y tabla destino

Se crean stg_lectura_raw para conservar el JSONB original, lectura_demo como tabla relacional y etl_log para la bitácora. El archivo JSONL se carga línea por línea mediante el script Python. Se eligió Python porque permite integrar la generación, carga y transformación en un único punto de entrada.

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Extensions
- Foreign Data Wrappers
- Languages
- Publications
- Schemas (1)
 - public
- Subscriptions
- postgres
- Login/Group Roles
- Tablespaces
- SolarDB - Lector
 - Databases (2)
 - SolarDB
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - postgres
 - Login/Group Roles
 - Tablespaces

SQL Statistics Dependencies Dependents Processes cargar_staging.sql* SolarDB/solar_lect...

SolarDB/postgres@PostgreSQL 18

No limit

Query Query History Scratch Pad

```
1 SELECT COUNT(*) AS total_lecturas
2 FROM lectura_demo;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

total_lecturas	bigint
1	288

Total rows: 1 Query complete 00:00:00.087 CRLF Ln 2, Col 19

21°

ESP LAA 7:08 p.m. 4/10/2026

Paso 3 · Transformación y carga

La transformación extrae los atributos con ->>, que devuelve texto, y luego realiza las conversiones a INT, TIMESTAMPTZ y NUMERIC. La clave primaria de lectura_demo identifica cada lectura y ON CONFLICT DO NOTHING evita duplicados.

```
INSERT INTO lectura_demo (
    dispositivo_id, ts, p_ac, irradiancia, temp_modulo, payload
)
SELECT
    (payload ->> 'device_id')::INT,
    (payload ->> 'ts')::TIMESTAMPTZ,
    (payload ->> 'p_ac')::NUMERIC(10,3),
    (payload ->> 'irradiancia')::NUMERIC(8,1),
    (payload ->> 'temp_modulo')::NUMERIC(5,1),
    payload
FROM stg_lectura_raw
WHERE id > :ultimo_id_staging
ON CONFLICT (dispositivo_id, ts) DO NOTHING;
```

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, including Schemas (1) with a public schema, and Databases (2) with SolarDB. The main pane shows a SQL query executed against SolarDB/postgres@PostgreSQL 18. The query is:

```

1 SELECT
2     dispositivo_id,
3     COUNT(*) AS cantidad_lecturas
4 FROM lectura_demo
5 GROUP BY dispositivo_id
6 ORDER BY dispositivo_id;

```

The Data Output pane shows the results of the query:

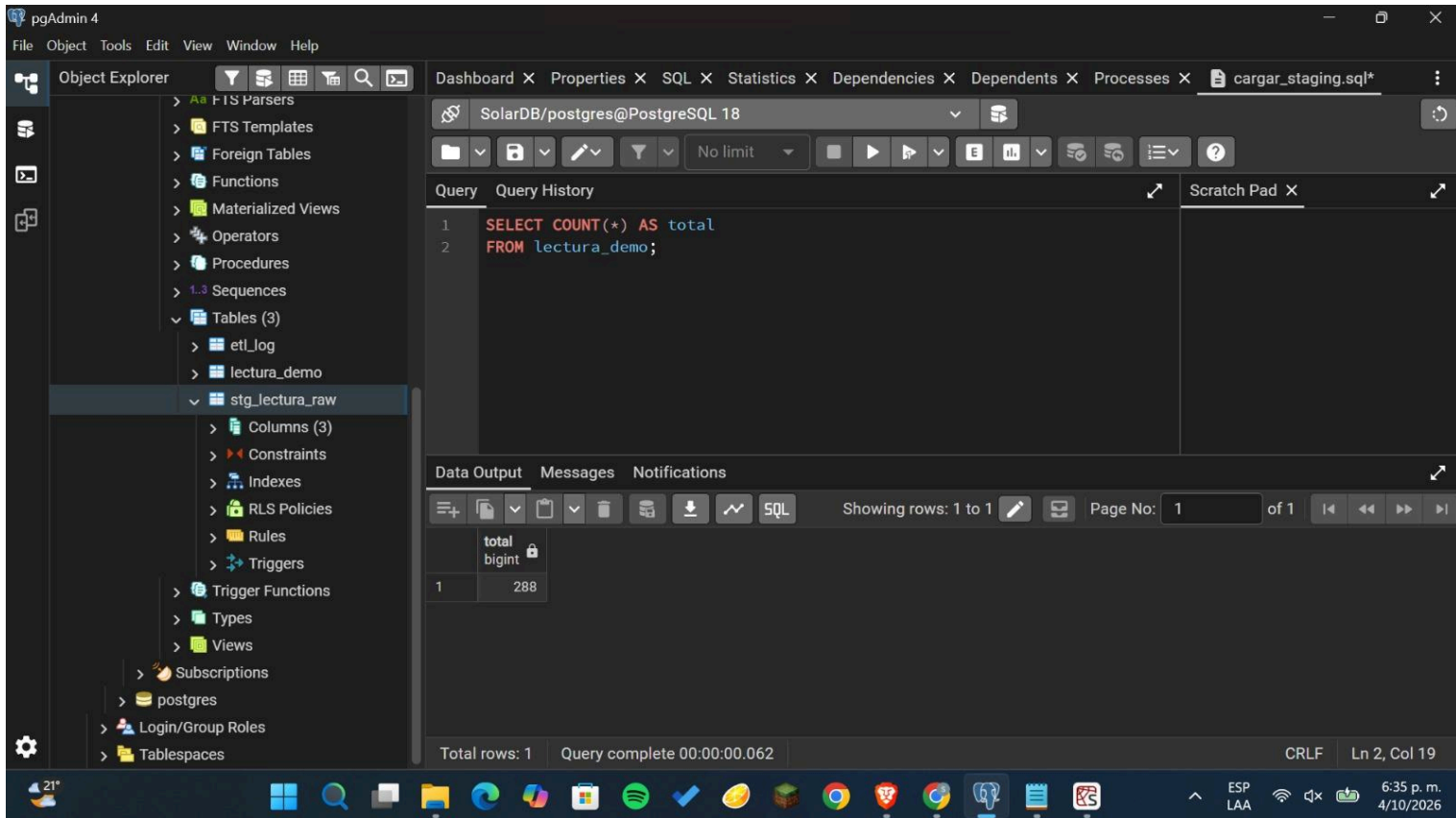
dispositivo_id	cantidad_lecturas
1	144
2	144

The status bar at the bottom indicates "Total rows: 2" and "Query complete 00:00:00.092".

Paso 4 · Prueba de idempotencia

Con el mismo archivo, la primera ejecución debe producir **288** filas en lectura_demo. Al ejecutar nuevamente el mismo flujo, el total debe permanecer en **288**. El staging sí puede aumentar porque conserva los mensajes recibidos, pero la tabla destino no duplica las lecturas gracias a la clave primaria y a ON CONFLICT DO NOTHING.

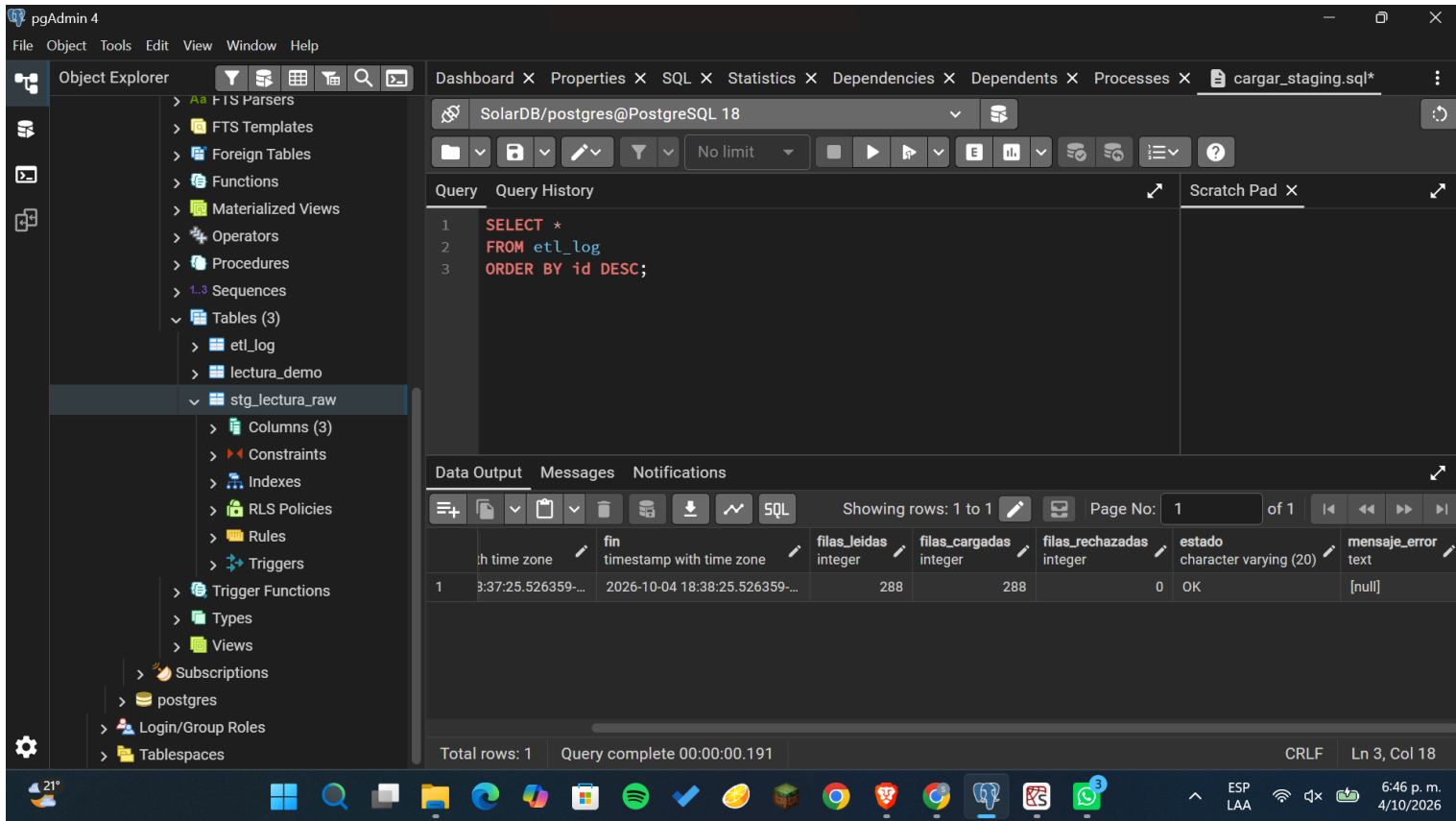
Ejecución	Filas nuevas esperadas	COUNT(*) lectura_demo
1	288	288
2	0 por conflicto	288



Paso 5 · Bitácora y automatización

Cada ejecución se registra en etl_log con inicio, fin, filas leídas, cargadas, rechazadas/conflictivas, estado y error. El punto de entrada único es python etl/run_etl.py. Para ejecutar cada hora en cron se puede usar 0 * * * *. En Windows se configura una tarea repetitiva cada hora apuntando al mismo script.

```
SELECT *
FROM etl_log
ORDER BY id DESC;
```



Paso 6 · Gobernanza aplicada

El archivo sql/03_rols.sql crea los roles solar_ingesta, solar_lector y solar_admin. Para la evidencia exigida se debe demostrar que solar_lector puede consultar pero no insertar.

```
GRANT SELECT ON TABLE lectura_demo TO solar_lector;

-- Prueba permitida:
SELECT * FROM lectura_demo LIMIT 5;
```

```
-- Prueba que debe ser rechazada:  
INSERT INTO lectura_demo  
(dispositivo_id, ts, p_ac, irradiancia, temp_modulo)  
VALUES (99, now(), 1.000, 500.0, 30.0);
```

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Extensions
- Foreign Data Wrappers
- Languages
- Publications
- Schemas (1)
 - public
- Subscriptions
- postgres
- Login/Group Roles
- Tablespaces
- SolarDB - Lector
 - Databases (2)
 - SolarDB
 - Casts
 - Catalogs
 - Event Triggers
 - Extensions
 - Foreign Data Wrappers
 - Languages
 - Publications
 - Schemas
 - Subscriptions
 - postgres
 - Login/Group Roles
 - Tablespaces

Dependencies X Dependents X Processes X cargar_staging.sql* X SolarDB/solar_lector@SolarDB - Lector* X

SolarDB/solar_lector@SolarDB - Lector

No limit

Query Query History

```
1 SELECT COUNT(*) AS total_lecturas  
2 FROM lectura_demo;
```

Scratch Pad X

Data Output Messages Notifications

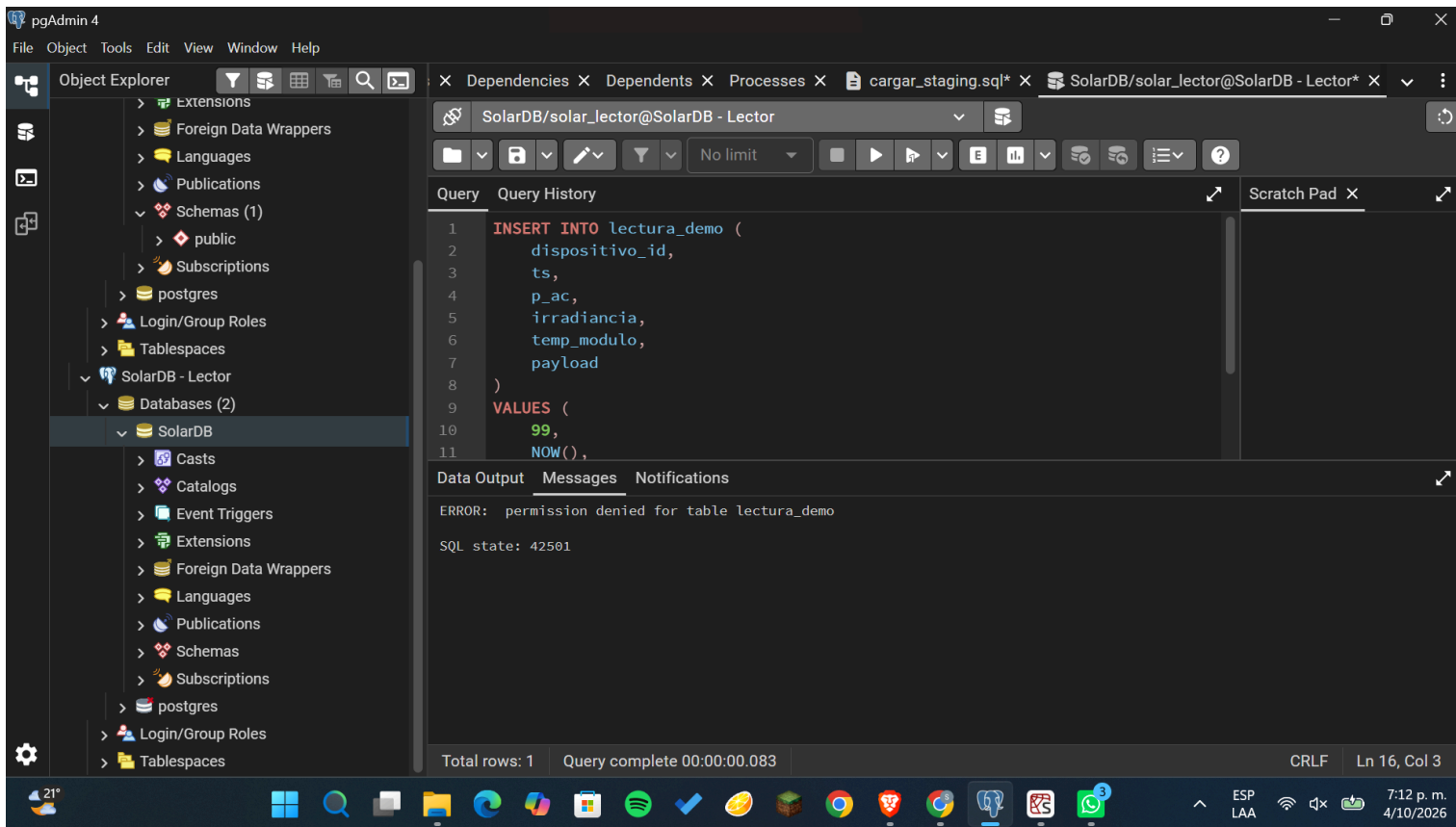
Showing rows: 1 to 1 Page No: 1 of 1

	total_lecturas bigint
1	288

Total rows: 1 Query complete 00:00:00.084 CRLF Ln 2, Col 19

21°

7:11 p. m.
4/10/2026



3. Parte C · Repositorio en GitHub

Nombre sugerido, siguiendo la guía: **solardb-apellido1-apellido2**. Reemplacen los apellidos por los de ambos integrantes. El repositorio debe ser público, el compañero debe aparecer como colaborador y ambos deben aportar mínimo tres commits con mensajes descriptivos.

Estructura incluida en el paquete

```
solardb-apellido1-apellido2/  
■■■ README.md  
■■■ .gitignore  
■■■ .env.example  
■■■ requirements.txt  
■■■ docs/  
■■■ data/  
■■■ lecturas.jsonl  
■■■ etl/  
■■■ simulador.py  
■■■ run_etl.py  
■■■ sql/  
■■■ 01_tablas.sql  
■■■ 02_carga.sql  
■■■ 03_rols.sql
```

<https://github.com/Sebastianceballos-507/solardb-ceballos-gazron-barrientos-lopez>

5. Referencias

- [1] DAMA International. DAMA Data Management Body of Knowledge (DAMA-DMBOK) y recursos sobre Data Management. <https://dama.org/learning-resources/dama-data-management-body-of-knowledge-dmbok/>
- [2] PostgreSQL 15 Documentation. Privileges. <https://www.postgresql.org/docs/15/ddl-priv.html>
- [3] PostgreSQL 15 Documentation. GRANT. <https://www.postgresql.org/docs/15/sql-grant.html>
- [4] PostgreSQL 15 Documentation. REVOKE. <https://www.postgresql.org/docs/15/sql-revoke.html>
- [5] Colombia, Departamento Administrativo de la Función Pública. Ley 1581 de 2012. <https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php/norma.php?i=49981>
- [6] PostgreSQL Documentation. JSON Functions and Operators. <https://www.postgresql.org/docs/current/functions-json.html>
- [7] PostgreSQL 15 Documentation. INSERT / ON CONFLICT. <https://www.postgresql.org/docs/15/sql-insert.html>
- [8] Apache Airflow Documentation. Core Concepts. <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/>
- [9] OASIS. MQTT Version 5.0. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [10] Modbus Organization. Specifications and Implementation Guides. <https://www.modbus.org/modbus-specifications> [11] OPC Foundation. OPC Unified Architecture – Security / Overview. <https://reference.opcfoundation.org/specs/OPC-10000-1/4.4.1>

